41039 Programming 1

Revision of Week 9: Exceptions and reading text



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. Catching exactly the invalid conversion

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
String[] values={"12","bad","7"};
for(String text:values) {
    try { System.out.println(Integer.parseInt(text)+1);
     }
    catch(NumberFormatException e) { System.out.println(
     "Invalid: "+text); }
}
```

Your answer

What is the complete output? Which line or rule decides it?

1. Catching exactly the invalid conversion

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
String[] values={"12","bad","7"};
for(String text:values) {
    try { System.out.println(Integer.parseInt(text)+1);
     }
    catch(NumberFormatException e) { System.out.println(
     "Invalid: "+text); }
}
```

Answer

```
13
Invalid: bad
8
```

The failed conversion jumps to the matching catch; the next loop iteration still runs.

2. Conversion errors in Python

Python 3; commit to an outcome before the reveal.

```
for text in ["12", "bad", "7"]:
    try:
        value = int(text)
    except ValueError:
        print("Invalid:", text)
    else:
        print(value + 1)
```

Your answer

What is the complete output? Which line or rule decides it?

2. Conversion errors in Python

Python 3; commit to an outcome before the reveal.

```
for text in ["12", "bad", "7"]:
    try:
        value = int(text)
    except ValueError:
        print("Invalid:", text)
    else:
        print(value + 1)
```

Answer

```
13
Invalid: bad
8
```

The else suite runs only for successfully parsed values; a ValueError produces the diagnostic.

3. Propagation and cleanup

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static void parse() { Integer.parseInt("x"); }
    public static void main(String[] args) {
        try {
            parse();
            System.out.println("after parse");
        } catch(NumberFormatException e) {
            System.out.println("caught");
        } finally {
            System.out.println("cleanup");
        }
        System.out.println("done");
    }
}
```

Your answer

What prints? Would adding throws to a method catch the failure? Which exception category is shown here?

3. Propagation and cleanup

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static void parse() { Integer.parseInt("x"); }
    public static void main(String[] args) {
        try {
            parse();
            System.out.println("after parse");
        } catch(NumberFormatException e) {
            System.out.println("caught");
        } finally {
            System.out.println("cleanup");
        }
        System.out.println("done");
    }
}
```

Answer

```
caught
cleanup
done
```

The line after parse is skipped. The matching catch runs, then finally, then the following statement. The failure is an unchecked NumberFormatException. A throws declaration allows propagation; it does not handle an exception. Checked exceptions such as IOException must be caught or declared.

4. Raise at the point a rule is broken

Python 3; commit to an outcome before the reveal.

```
def mark(text):
    value = int(text)
    if not 0 <= value <= 100:
        raise ValueError("mark out of range")
    return value
try:
    print(mark("120"))
except ValueError as error:
    print(error)
finally:
    print("checked")
```

Your answer

What is the complete output? Which line or rule decides it?

4. Raise at the point a rule is broken

Python 3; commit to an outcome before the reveal.

```
def mark(text):
    value = int(text)
    if not 0 <= value <= 100:
        raise ValueError("mark out of range")
    return value
try:
    print(mark("120"))
except ValueError as error:
    print(error)
finally:
    print("checked")
```

Answer

```
mark out of range
checked
```

The conversion succeeds, but the range check raises before returning a mark.

5. BufferedReader stops at null, not ready()

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
try(BufferedReader reader=new BufferedReader(new
     StringReader("tea\n\ncoffee"))) {
    String line;
    int count=0;
    while((line=reader.readLine())!=null) {
        count++;
        System.out.println(count+": "+line);
    }
} catch(IOException e) { System.out.println("Read failed
     "); }
```

Your answer

What is the complete output? Which line or rule decides it?

5. BufferedReader stops at null, not ready()

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
try(BufferedReader reader=new BufferedReader(new
     StringReader("tea\n\ncoffee"))) {
    String line;
    int count=0;
    while((line=reader.readLine())!=null) {
        count++;
        System.out.println(count+": "+line);
    }
} catch(IOException e) { System.out.println("Read failed
     "); }
```

Answer

```
1: tea
2:
3: coffee
```

The reader returns an empty string for the middle line, and null only after coffee.

6. Line input has a different contract

Python 3; commit to an outcome before the reveal.

```
from io import StringIO
with StringIO("tea\n\ncoffee\n") as source:
    for line_number, line in enumerate(source, 1):
        print(f"{line_number}: {line.rstrip(chr(10))}")
```

Your answer

What is the complete output? Which line or rule decides it?

6. Line input has a different contract

Python 3; commit to an outcome before the reveal.

```
from io import StringIO
with StringIO("tea\n\ncoffee\n") as source:
    for line_number, line in enumerate(source, 1):
        print(f"{line_number}: {line.rstrip(chr(10))}")
```

Answer

```
1: tea
2:
3: coffee
```

A blank line still exists; stripping its newline does not remove the line from iteration.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: The exception path for the middle record

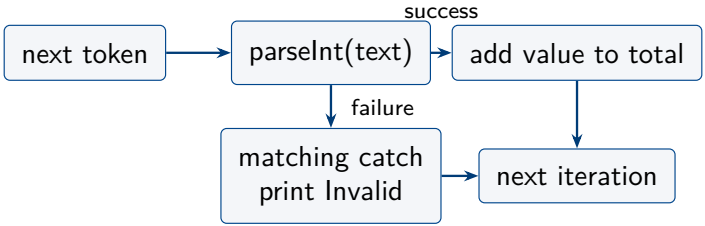
```
int total=0;
for(String text:new String[]{"12","bad","7"}) {
    try { total+=Integer.parseInt(text); }
    catch(NumberFormatException e) { System.out.println(
     "Invalid"); }
}
System.out.println(total);
```

Four minutes

Draw the flow and responsibilities after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Course website: Week 9. Complete programs accompany this deck.

Draw it: one answer



Output

```
Invalid
19
```

For bad, parsing throws before addition. The catch runs once, total remains 12, and the following iteration adds 7.

Course website: Week 9. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

Print Invalid for the bad token, then the count and sum of valid tokens: 2 19.

```java
public class Broken {
    public static void main(String[] args) {
        String[] input={"12","bad","7"};
        int total=0,count=0;
        try {
            for(String text:input) {
                count++;
                total+=Integer.parseInt(text);
            }
        } catch(NumberFormatException e) {
            System.out.println("Invalid");
        }
        System.out.println(count+" "+total);
    }
}
```

Spot the errors 1: answers

The repairs

- **L5:** Handling outside the entire loop stops later records after the first failure. Put handling inside each iteration.
- **L7:** Count successful records after conversion succeeds.

Course website: Week 9. Complete programs accompany this deck.

✗ Original program

```
Invalid
2 12
```

✓ After the repairs

```
Invalid
2 19
```

Spot the errors 2: Python (5 min)

Contract

Report each invalid number, continue to later lines, and print the total of valid input. Use a context manager for the reader.

```
from io import StringIO
source=StringIO("12\nbad\n7\n")
for line in source:
    total=0
    try:
        total+=int(line)
    except FileNotFoundError:
        print("Invalid")
print(total)
source.close()
```

Spot the errors 2: answers

The repairs

- **L4:** Initialise the accumulator before the loop.
- **L7:** int(line) raises ValueError for bad text; catch that specific failure.

✗ Original program

```
(no standard output)
ValueError: invalid literal for
    int() with base 10: 'bad\n'
```

✓ After the repairs

```
Invalid
19
```

Course website: Week 9. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: A caught failure and finally

```
try {
    System.out.println("A");
    Integer.parseInt("bad");
    System.out.println("B");
} catch(NumberFormatException e) {
    System.out.println("C");
} finally { System.out.println("D"); }
System.out.println("E");
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 9. Complete programs accompany this deck.

Predict 1: answer

```
A
C
D
E
```

Why

The failed conversion skips B. The matching catch prints C, finally prints D, and the handled exception allows E.

Course website: Week 9. Complete programs accompany this deck.

Predict 2: Finally before return or propagation

```
def parse(text):
    try:
        return int(text)
    finally:
        print("checked",text)
for text in ["4","x","2"]:
    try: print(parse(text))
    except ValueError: print("bad")
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 9. Complete programs accompany this deck.

Predict 2: answer

```
checked 4
4
checked x
bad
checked 2
2
```

Why

finally runs when parse returns and when its exception propagates. The caller handles x, then continues with 2.

Course website: Week 9. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

Continue after an invalid record

Read n then n lines. A line is integer text or an invalid word. Print Invalid for each failed conversion, keep processing, and finish with Total: N. Keep the try block around the conversion that may fail.

Example input

```
4
12
bad
-2
7
```

Expected output

```
Invalid
Total: 17
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution

```java
import java.util.Scanner;
Scanner sc=new Scanner(System.in);
int n=Integer.parseInt(sc.nextLine());
int total=0;
for(int i=0;i<n;i++) {
    try { total+=Integer.parseInt(sc.nextLine()); }
    catch(NumberFormatException e) { System.out.println("Invalid"); }
}
System.out.println("Total: "+total);
```

Course website: Week 9. Complete programs accompany this deck.

Code it: Python solution

```python
n=int(input())
total=0
for _ in range(n):
    try: total+=int(input())
    except ValueError: print("Invalid")
print(f"Total: {total}")
```

Course website: Week 9. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`4 / 12 / bad / -2 / 7`
Output: `Invalid / Total: 17`

Test 2
`0`
Output: `Total: 0`

Test 3
`2 / x / y`
Output: `Invalid / Invalid / Total: 0`

Test 4
`2 / 0 / 100`
Output: `Total: 100`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Trace the first statement skipped by an exception.
- Distinguish handling from a throws declaration.
- Recognise blank lines, EOF and token boundaries.
- Close a reader while preserving the intended recovery policy.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.
Course website: Week 9. Complete programs accompany this deck.